

JavaScript Revision Roadmap — Basic → Advanced

PART 1 — JavaScript Fundamentals

- What is JavaScript?
- JavaScript Engine
- ECMAScript
- JavaScript vs Java
- `use strict`

Variables & Values

- Variables
- var
- let
- const
- Reassignment
- Declaration vs Initialization
- Primitive vs Reference values

Data Types

- String
- Number
- BigInt
- Boolean
- Undefined
- Null
- Symbol
- Object
- typeof
- typeof null quirk

Operators

- Arithmetic
- Assignment
- Comparison
- Logical
- Unary

- Ternary
- Nullish Coalescing (??)
- Optional Chaining (?.)
- ?? vs ||
- == vs ===

Type Conversion

- String()
- Number()
- Boolean()
- parseInt()
- parseFloat()
- Implicit coercion
- Explicit coercion

Truthy / Falsy

- Truthy values
- Falsy values
- Boolean()
- Condition evaluation

PART 2 — Conditions & Decision Making

- if
- else
- else if
- Nested conditions
- Ternary operator
- switch
- break
- Logical conditions
- Short-circuit evaluation

Practice:

- Login system
- Age checker
- Grade calculator

Role-based access
Password validation

PART 3 — Loops & Iteration

এইটা খুব important, কারণ পরে Arrays-এর ভিতরের logic বুঝতে লাগবে।

- for loop
- while loop
- do...while
- break
- continue
- Nested loops
- for...of
- for...in

Understand the difference

```
for
while
do...while
for...of
for...in
```

Practice:

- Print 1–100
- Even / odd numbers
- Sum of numbers
- Multiplication table
- Find largest number
- Reverse loop
- Nested pattern
- Search value

PART 4 — Functions

এটা JavaScript-এর **core foundation**।

- Function declaration
- Function expression
- Arrow function

- Parameters
- Arguments
- Return
- Default parameters
- Multiple parameters
- Rest parameters
- Callback functions
- Higher-order functions

Then:

- Function scope
- Lexical scope
- Block scope
- Global scope
- Local scope
- Nested functions

Then advanced:

- Hoisting
- Closure
- IIFE
- Recursion
- Pure functions
- Side effects

Practice:

```
createUser()  
calculateTotal()  
validateEmail()  
isAdmin()  
generateToken()  
filterUsers()
```

PART 5 — Arrays

- Array creation
- Index
- length
- Access

- Update
- Add
- Remove

Basic methods

- push()
- pop()
- shift()
- unshift()
- slice()
- splice()

Most important

- forEach()
- map()
- filter()
- find()
- findIndex()
- some()
- every()
- reduce()
- sort()
- includes()
- indexOf()

Array concepts

- Mutable vs immutable operations
- Chaining methods
- Array destructuring
- Spread operator
- Rest operator
- Nested arrays

Practice:

- Product filtering
- User searching
- Cart total
- Sorting products

- ❑ Find highest price
 - ❑ Group/filter data
-

PART 6 — Objects

- ❑ Object creation
- ❑ Properties
- ❑ Values
- ❑ Dot notation
- ❑ Bracket notation
- ❑ Add property
- ❑ Update property
- ❑ Delete property
- ❑ Nested objects
- ❑ Object methods

Important methods

- ❑ `Object.keys()`
- ❑ `Object.values()`
- ❑ `Object.entries()`
- ❑ `Object.assign()`
- ❑ `Object.hasOwn()`

Modern object concepts

- ❑ Object destructuring
- ❑ Nested destructuring
- ❑ Spread operator
- ❑ Rest operator
- ❑ Shorthand properties
- ❑ Computed properties

Important

- ❑ Reference vs copy
 - ❑ Shallow copy
 - ❑ Deep copy
 - ❑ `structuredClone()`
-

PART 7 — ES6+ / Modern JavaScript

এখানে modern JS-এর major features consolidate করবে।

- ❑ let / const
- ❑ Arrow functions
- ❑ Template literals
- ❑ Destructuring
- ❑ Spread
- ❑ Rest
- ❑ Default parameters
- ❑ Enhanced object literals
- ❑ Optional chaining
- ❑ Nullish coalescing
- ❑ Modules

Modules

- ❑ export
- ❑ import
- ❑ Named export
- ❑ Default export
- ❑ CommonJS
- ❑ ES Modules

PART 8 — Advanced Functions

এখানে JavaScript-এর actual power বুঝতে শুরু করবে।

- ❑ First-class functions
- ❑ Callback
- ❑ Higher-order functions
- ❑ Function as argument
- ❑ Function as return value
- ❑ Closures
- ❑ Lexical environment
- ❑ Scope chain
- ❑ IIFE
- ❑ Recursion

Must understand

Function
↓
Scope
↓
Lexical Scope
↓
Closure
↓
Callback
↓
Higher Order Function

🟡 PART 9 — `this` & Object-Oriented JavaScript

- What is `this``
- Global context
- Object method
- Regular function
- Arrow function
- `call()`
- `apply()`
- `bind()`

Then:

- Constructor functions
- Prototype
- Prototype chain
- Class
- Constructor
- `extends`
- `super`
- Encapsulation concept
- Inheritance
- Polymorphism

🟡 PART 10 — Asynchronous JavaScript

এই sectionটা **Node.js**-এর জন্য অত্যন্ত important।

First:

- Synchronous code
- Asynchronous code
- Blocking
- Non-blocking
- setTimeout()
- setInterval()

Then:

- Callback
- Callback hell
- Promise
- Promise states
- resolve
- reject
- .then()
- .catch()
- .finally()

Then:

- async
- await
- try/catch
- Error handling

PART 11 — Event Loop

এটা **must-learn before serious Node.js** ।

- JavaScript Runtime
- Call Stack
- Web APIs / Runtime APIs
- Callback Queue
- Microtask Queue
- Event Loop
- Macrotask
- Microtask

Must understand:

```
Call Stack
  ↓
Web/Runtime APIs
  ↓
Callback Queue
  ↓
Event Loop
  ↓
Call Stack
```

And:

```
Promise callbacks
  ↓
Microtask Queue

setTimeout callback
  ↓
Task/Macrotask Queue
```

Practice output prediction:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Expected:

```
A
D
C
B
```

PART 12 — Promises & Async Patterns

- Promise chaining
- Promise.all()
- Promise.allSettled()
- Promise.race()
- Promise.any()
- Sequential async operations
- Parallel async operations
- Error propagation
- Async error handling

Important mental model:

```
Promise
  ↓
.then()
  ↓
async/await
  ↓
Concurrent operations
  ↓
Real API / Database work
```

PART 13 — Error Handling

- Syntax Error
- Runtime Error
- Logical Error
- Error object
- throw
- try
- catch
- finally
- Custom Error
- Error propagation

Practice:

```
try {
  // risky operation
```

```
} catch (error) {  
  console.error(error.message);  
}
```

PART 14 — DOM & Browser JavaScript

যেহেতু তুমি React already করেছ, এটাকে quick revision হিসেবে করলেই হবে।

- DOM
- querySelector()
- querySelectorAll()
- getElementById()
- createElement()
- append()
- remove()
- classList
- attributes
- textContent
- innerHTML

Events:

- click
- submit
- input
- change
- keydown
- event object
- event bubbling
- event capturing
- event delegation
- preventDefault()

PART 15 — HTTP + Fetch

Node backend-এর আগে এই অংশটা strong করো।

- HTTP
- Request
- Response
- HTTP Methods
- GET

- POST
- PUT
- PATCH
- DELETE
- Status Codes
- Headers
- Body
- JSON

Then:

- fetch()
- Request
- Response
- response.json()
- response.ok
- HTTP errors
- Network errors

PART 16 — Deep JavaScript Concepts

এগুলোতে ঢুকলে JS-এর ভিতরের mechanics বুঝবে।

- Execution Context
- Global Execution Context
- Function Execution Context
- Call Stack
- Lexical Environment
- Environment Record
- Scope Chain
- Hoisting
- Temporal Dead Zone
- Closures
- Garbage Collection
- Memory management

Then:

- Primitive vs Reference
- Stack vs Heap conceptual model
- Shallow copy
- Deep copy

- Immutability
- Mutation

PART 17 — Advanced JavaScript

- Prototype
- Prototype Chain
- Classes
- Iterators
- Iterables
- Generators
- Symbol
- Map
- Set
- WeakMap
- WeakSet

Advanced async:

- Async iterators
- for await...of
- AbortController
- Race conditions
- Concurrency
- Sequential vs parallel execution

PART 18 — JavaScript Modules & Runtime

- ES Modules
- CommonJS
- import
- export
- require()
- module.exports
- package.json
- npm
- node_modules
- package-lock.json

Then understand:

Browser JavaScript



JavaScript Engine

Node.js



V8 + Node Runtime APIs



File System

HTTP

Networking

Streams

Process



PART 19 — Node.js Preparation

JavaScript revision শেষ করার পরে:

- ☐ Event Loop
- ☐ Async/Await
- ☐ Promise
- ☐ Error Handling
- ☐ Modules
- ☐ JSON
- ☐ Objects
- ☐ Functions
- ☐ Closures
- ☐ HTTP
- ☐ Fetch
- ☐ Event-driven programming

Then:

JavaScript



Node.js Runtime



HTTP Server



Express.js



REST API
↓
Database
↓
Authentication
↓
Production API

Your Revision Order

তোমার জন্য আমি এই **exact order**-টাই **follow** করতাম:

01. Fundamentals
↓
02. Variables & Data Types
↓
03. Operators & Type Conversion
↓
04. Conditions
↓
05. Loops
↓
06. Functions
↓
07. Scope
↓
08. Hoisting
↓
09. Closures
↓
10. Arrays
↓
11. Objects
↓
12. Destructuring
↓
13. Spread / Rest
↓
14. ES6+
↓
15. this

↓
16. Prototype / Classes
↓
17. Callbacks
↓
18. Promises
↓
19. Async / Await
↓
20. Error Handling
↓
21. Event Loop
↓
22. Concurrency
↓
23. DOM / Events
↓
24. HTTP / JSON / Fetch
↓
25. Modules / npm
↓
26. Deep JS Internals
↓
27. Node.js

Revision Rule

প্রতিটা topic-এ এই **5-step system** use করো:

1 Understand
↓
2 Write from memory
↓
3 Solve 3–5 problems
↓
4 Build a tiny feature
↓
5 Explain without looking

Example — `Array.map()` :

Understand map()



Write map() yourself



Solve 5 problems



Build product transformation



Explain:

"map returns a new array..."

তোমার লক্ষ্য হওয়া উচিত: